```
print("Dhwaj Jain S102  DL Expt 4")
```

```
Dhwaj Jain S102  DL Expt 4
```

```
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

# 1. Create a labeled dataset of 20 sentences
sentences = [
    "I love this product, it's amazing!", "This is a fantastic movie.",
    "What a wonderful day it is.", "I'm so happy with the results.",
    "Excellent service and great quality.", "The food was delicious.",
    "Such a beautiful view.", "Highly recommend this place.",
    "Feeling very positive today.", "This made my day.",
    "I dislike this product, it's terrible.", "This is a boring movie.",
    "What a terrible day it is.", "I'm so sad with the results.",
    "Poor service and bad quality.", "The food was disgusting.",
    "Such an ugly view.", "Would not recommend this place.",
    "Feeling very negative today.", "This ruined my day."
]

# Labels: 1 for positive, 0 for negative/neutral
labels = np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])

print(f"Number of sentences: {len(sentences)}")
print(f"Number of labels: {len(labels)}")
```

```
Number of sentences: 20
Number of labels: 20
```

```
# 2. Text preprocessing: Tokenization and Padding

# Initialize tokenizer
# num_words: the maximum number of words to keep, based on word frequency.
# Only the most common `num_words` words will be kept.
tokenizer = Tokenizer(num_words=None, oov_token='<unk>')
tokenizer.fit_on_texts(sentences)

# Convert text to sequences of integers
sequences = tokenizer.texts_to_sequences(sentences)

# Determine the maximum sequence length for padding
max_sequence_length = max([len(x) for x in sequences])
print(f"Max sequence length: {max_sequence_length}")

# Pad sequences to ensure uniform input size
padded_sequences = pad_sequences(sequences, maxlen=max_sequence_length, padding='post')

print("Original sequences (first 2):")
print(sequences[:2])
print("Padded sequences (first 2):")
print(padded_sequences[:2])

# Vocabulary size (add 1 for the OOV token)
vocab_size = len(tokenizer.word_index) + 1
print(f"Vocabulary size: {vocab_size}")
```

```
Max sequence length: 6
Original sequences (first 2):
[[7, 31, 2, 8, 9, 32], [2, 4, 3, 33, 10]]
Padded sequences (first 2):
[[ 7 31  2  8  9 32]
 [ 2  4  3 33 10  0]]
Vocabulary size: 55
```

```
# 3. Build the Simple RNN model

# Embedding dimension for word vectors
embedding_dim = 100 # This can be tuned

model = Sequential([
```

```
    Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_sequence_length),
    SimpleRNN(32), # Simple RNN layer with 32 units
    Dense(1, activation='sigmoid') # Output dense layer with sigmoid activation for binary classification
])

# 4. Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecat
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

```
# 5. Train the model
# Given the small dataset, we'll train directly on it.
# For a real-world scenario, you would split into training and validation sets.

history = model.fit(
    padded_sequences,
    labels,
    epochs=20, # Number of epochs can be adjusted
    verbose=1
)

print("\nModel training complete.")
```

```
Epoch 1/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.5500 - loss: 0.6909
Epoch 2/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step - accuracy: 0.6000 - loss: 0.6761
Epoch 3/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.8500 - loss: 0.6617
Epoch 4/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.9000 - loss: 0.6475
Epoch 5/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 143ms/step - accuracy: 1.0000 - loss: 0.6330
Epoch 6/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - accuracy: 1.0000 - loss: 0.6183
Epoch 7/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - accuracy: 1.0000 - loss: 0.6030
Epoch 8/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step - accuracy: 1.0000 - loss: 0.5871
Epoch 9/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 109ms/step - accuracy: 1.0000 - loss: 0.5705
Epoch 10/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 1.0000 - loss: 0.5531
Epoch 11/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step - accuracy: 1.0000 - loss: 0.5349
Epoch 12/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 1.0000 - loss: 0.5159
Epoch 13/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step - accuracy: 1.0000 - loss: 0.4961
Epoch 14/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 1.0000 - loss: 0.4754
Epoch 15/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - accuracy: 1.0000 - loss: 0.4539
Epoch 16/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 143ms/step - accuracy: 1.0000 - loss: 0.4317
Epoch 17/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - accuracy: 1.0000 - loss: 0.4088
Epoch 18/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step - accuracy: 1.0000 - loss: 0.3855
Epoch 19/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 1.0000 - loss: 0.3617
Epoch 20/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step - accuracy: 1.0000 - loss: 0.3376

Model training complete.
```

```
# 6. Evaluate the model

loss, accuracy = model.evaluate(padded_sequences, labels, verbose=0)
print(f"\nAccuracy on the dataset: {accuracy*100:.2f}%")

# You can also make predictions
predictions = (model.predict(padded_sequences) > 0.5).astype(int)

print("\nExample Predictions (first 5):")
for i in range(5):
    print(f"Sentence: '{sentences[i]}' -> Actual: {labels[i]}, Predicted: {predictions[i][0]}")
```

```
Accuracy on the dataset: 100.00%
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 253ms/step

Example Predictions (first 5):
Sentence: 'I love this product, it's amazing!' -> Actual: 1, Predicted: 1
Sentence: 'This is a fantastic movie.' -> Actual: 1, Predicted: 1
Sentence: 'What a wonderful day it is.' -> Actual: 1, Predicted: 1
Sentence: 'I'm so happy with the results.' -> Actual: 1, Predicted: 1
Sentence: 'Excellent service and great quality.' -> Actual: 1, Predicted: 1
```

```
# 4. Evaluate the LSTM model

lstm_loss, lstm_accuracy = lstm_model.evaluate(padded_sequences, labels, verbose=0)
print(f"\nAccuracy of Simple LSTM model on the dataset: {lstm_accuracy*100:.2f}%")

# Make predictions with LSTM model
lstm_predictions = (lstm_model.predict(padded_sequences) > 0.5).astype(int)

print("\nExample Predictions (first 5) from LSTM Model:")
for i in range(5):
    print(f"Sentence: '{sentences[i]}' -> Actual: {labels[i]}, Predicted: {lstm_predictions[i][0]}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_184/1890428854.py in <cell line: 0>()
      1 # 4. Evaluate the LSTM model
      2
----> 3 lstm_loss, lstm_accuracy = lstm_model.evaluate(padded_sequences, labels, verbose=0)
      4 print(f"\nAccuracy of Simple LSTM model on the dataset: {lstm_accuracy*100:.2f}%")
      5

NameError: name 'lstm_model' is not defined
```